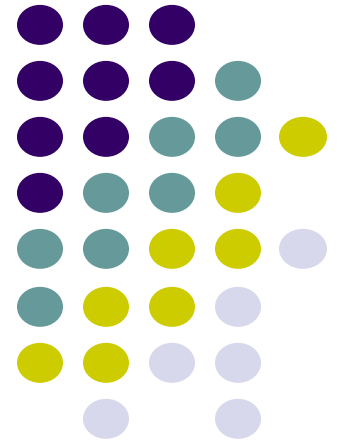


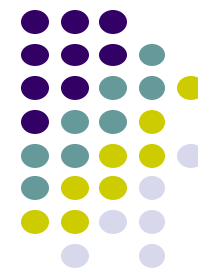
ErSE222: Machine learning in Geoscience

May. 7th, 2025

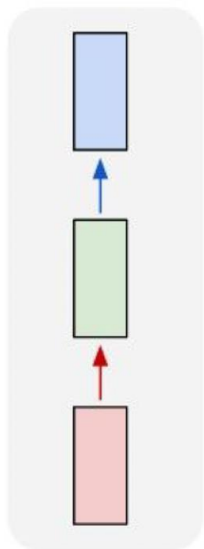
Tariq Alkhalifah and Omar Saad



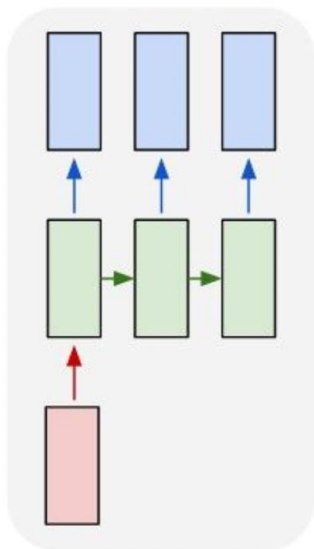
RNN



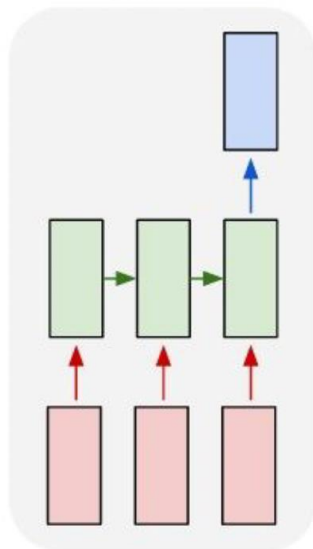
one to one



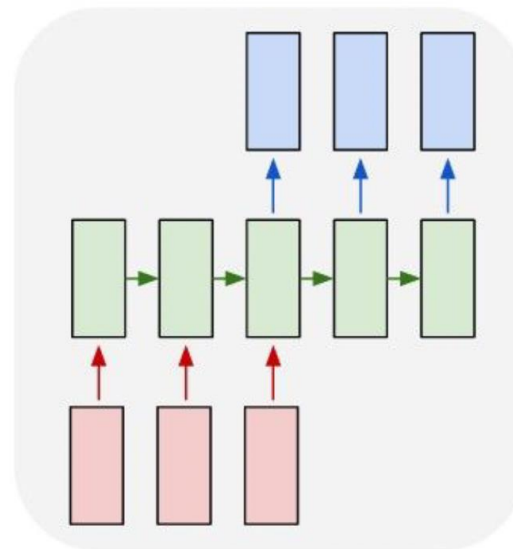
one to many



many to one



many to many



many to many

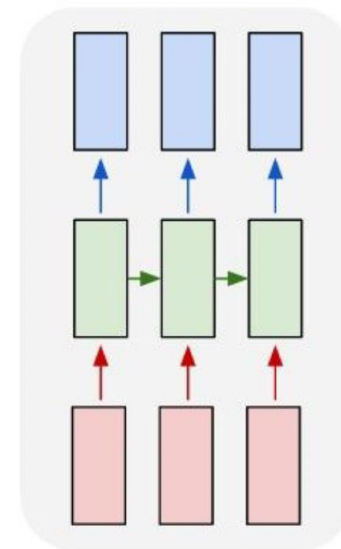
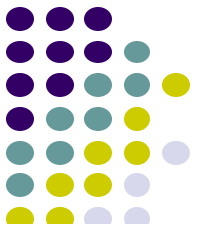


Image Captioning using spatial features



Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

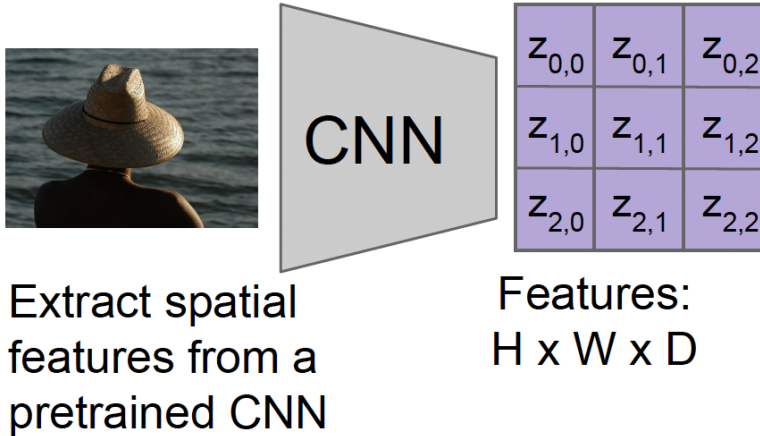
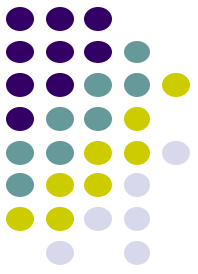


Image Captioning using spatial features



Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

where $\mathbf{z}$ is spatial CNN features

$f_w(\cdot)$ is an MLP

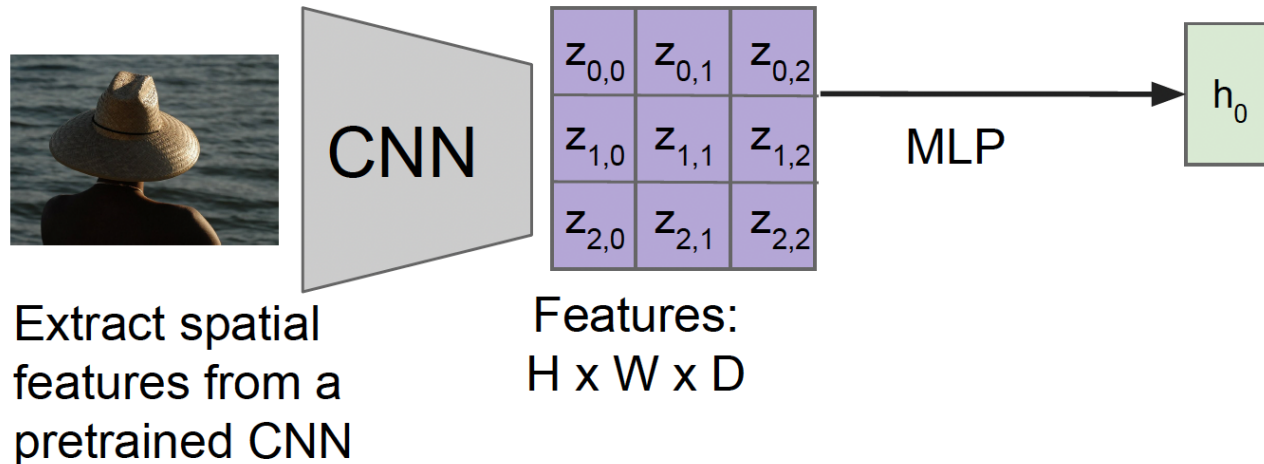
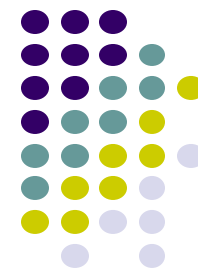


Image Captioning using spatial features



Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

where $\mathbf{z}$ is spatial CNN features

$f_w(\cdot)$ is an MLP

Decoder: $y_t = g_v(y_{t-1}, h_{t-1}, c)$

where context vector c is often $c = h_0$

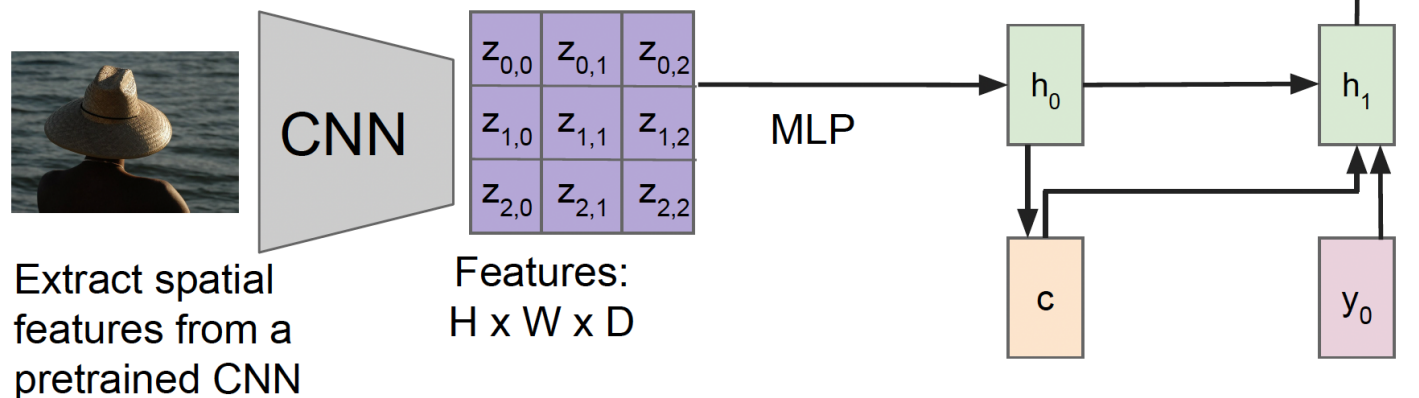
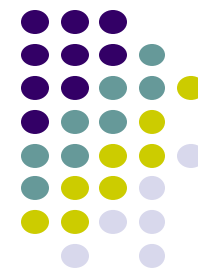


Image Captioning using spatial features



Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $h_0 = f_w(\mathbf{z})$

where $\mathbf{z}$ is spatial CNN features

$f_w(\cdot)$ is an MLP



CNN

Extract spatial
features from a
pretrained CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
 $H \times W \times D$

MLP

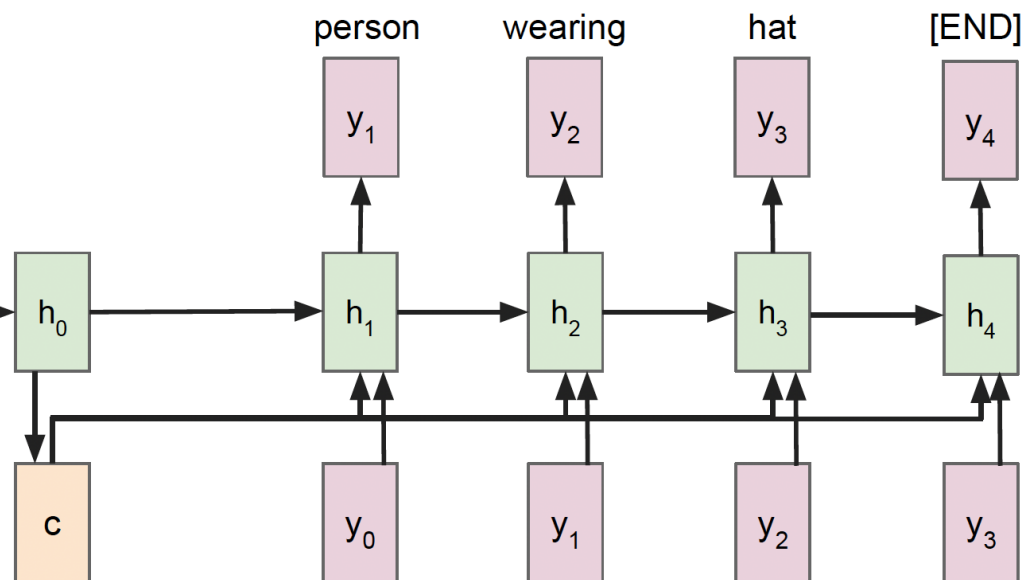
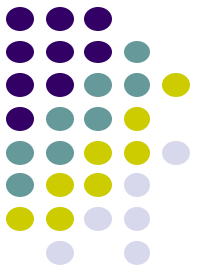


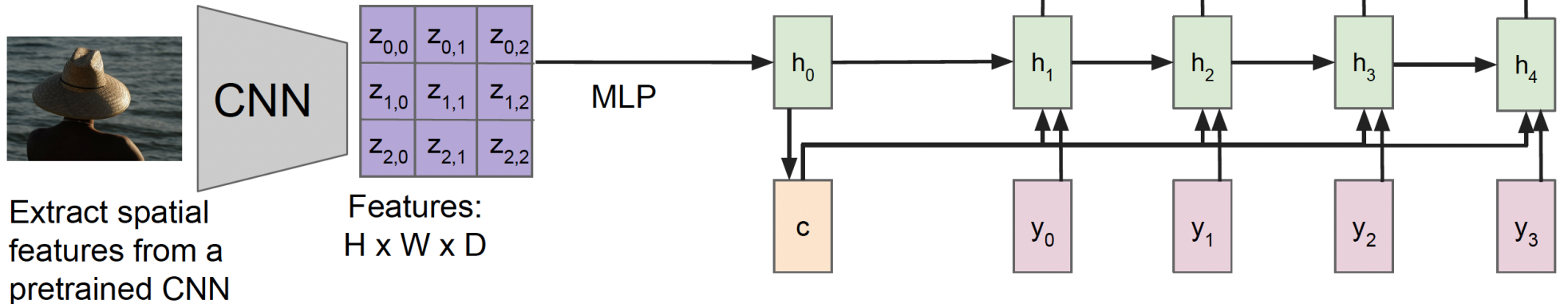
Image Captioning using spatial features



Problem: Input is "bottlenecked" through c

- Model needs to encode everything it wants to say within c

This is a problem if we want to generate really long descriptions? 100s of words long



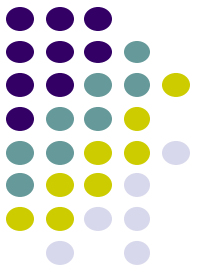


Image Captioning using spatial features

Attention idea: New context vector at every time step.

Each context vector will attend to different image regions

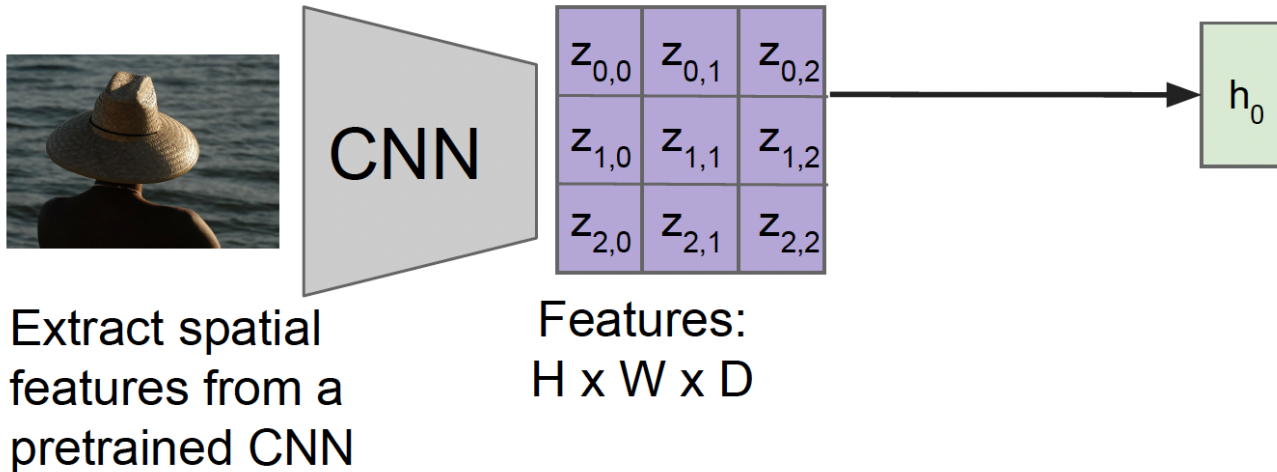




Image Captioning using spatial features and Attention

Compute alignments scores (scalars):

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$f_{att}(\cdot)$ is an MLP

Alignment scores:

H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$



CNN

Extract spatial features from a pretrained CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

h_0

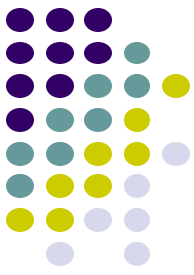


Image Captioning using spatial features and Attention

Compute alignments scores (scalars):

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$f_{att}(\cdot)$ is an MLP

Alignment scores:
H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$

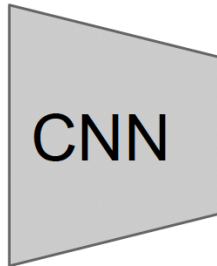
Attention:
H x W

$a_{1,0,0}$	$a_{1,0,1}$	$a_{1,0,2}$
$a_{1,1,0}$	$a_{1,1,1}$	$a_{1,1,2}$
$a_{1,2,0}$	$a_{1,2,1}$	$a_{1,2,2}$

Normalize to get
attention weights:

$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$

$0 < a_{t,i,j} < 1$,
attention values sum to 1



Extract spatial
features from a
pretrained CNN

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

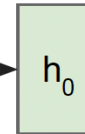
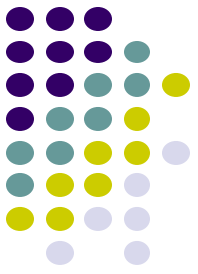


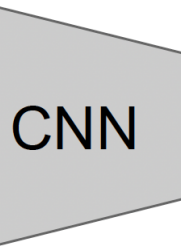
Image Captioning using spatial features and Attention



Compute alignments scores (scalars):

$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$f_{att}(\cdot)$ is an MLP



Extract spatial features from a pretrained CNN

Alignment scores:
H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$

Attention:
H x W

$a_{1,0,0}$	$a_{1,0,1}$	$a_{1,0,2}$
$a_{1,1,0}$	$a_{1,1,1}$	$a_{1,1,2}$
$a_{1,2,0}$	$a_{1,2,1}$	$a_{1,2,2}$

Normalize to get attention weights:

$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$

$0 < a_{t,i,j} < 1$,
attention values sum to 1

Compute context vector:

$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

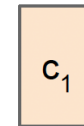
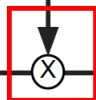
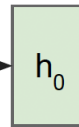
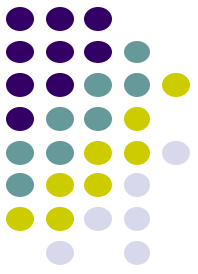


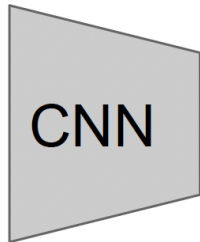
Image Captioning using spatial features and Attention



$$e_{t,i,j} = f_{att}(h_{t-1}, z_{i,j})$$

$$a_{t,:,:) = \text{softmax}(e_{t,:,:})$$

$$c_t = \sum_{i,j} a_{t,i,j} z_{t,i,j}$$



Extract spatial features from a pretrained CNN

Alignment scores:
H x W

$e_{1,0,0}$	$e_{1,0,1}$	$e_{1,0,2}$
$e_{1,1,0}$	$e_{1,1,1}$	$e_{1,1,2}$
$e_{1,2,0}$	$e_{1,2,1}$	$e_{1,2,2}$

Attention:
H x W

$a_{1,0,0}$	$a_{1,0,1}$	$a_{1,0,2}$
$a_{1,1,0}$	$a_{1,1,1}$	$a_{1,1,2}$
$a_{1,2,0}$	$a_{1,2,1}$	$a_{1,2,2}$

$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

Features:
H x W x D

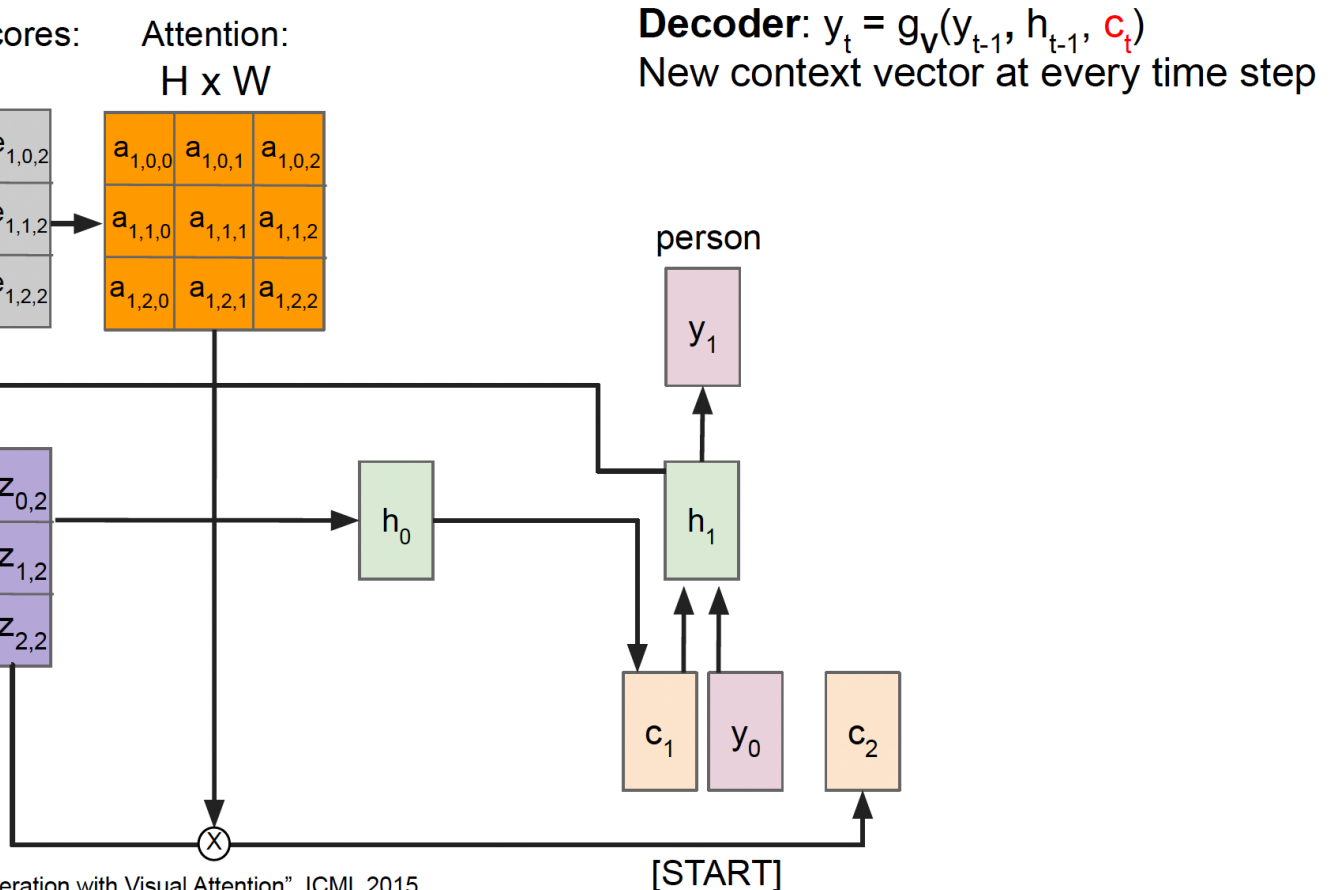
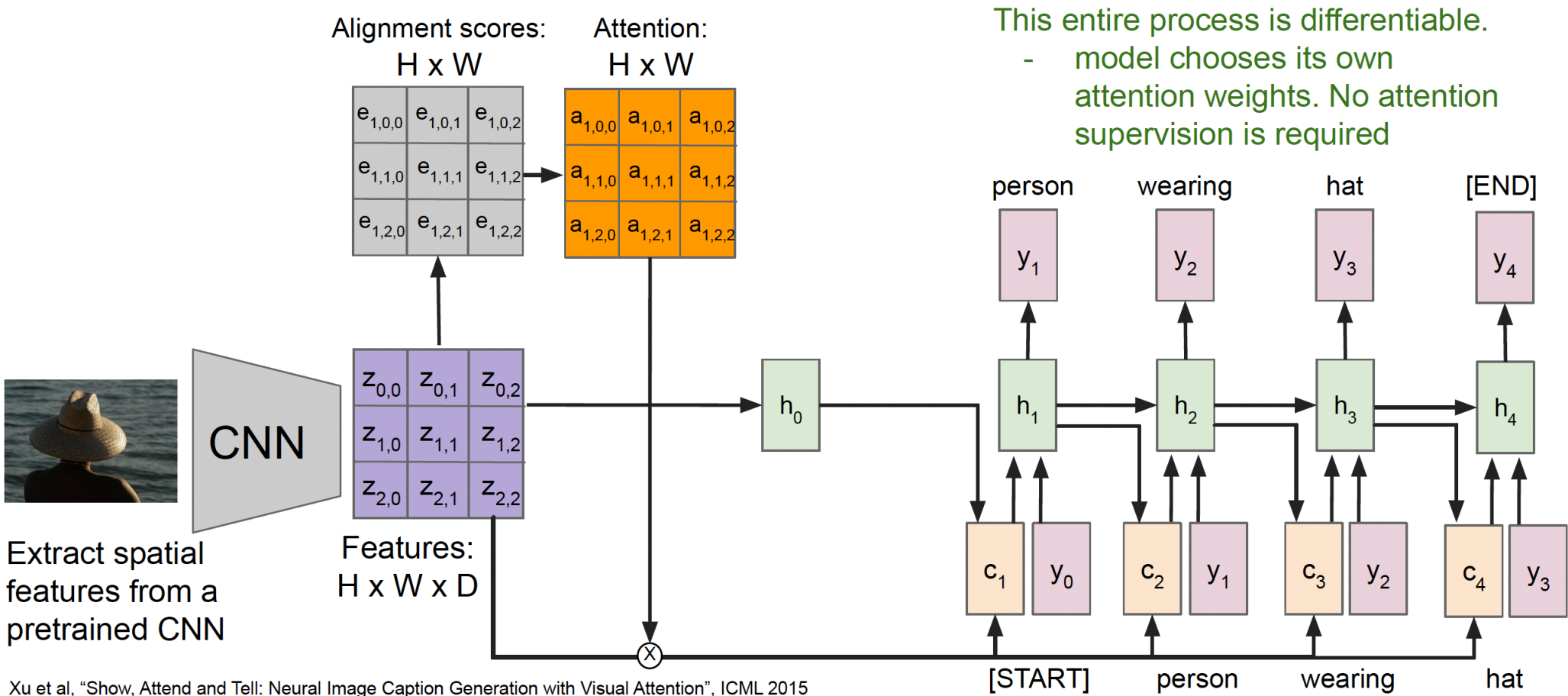
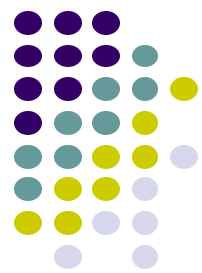


Image Captioning using spatial features and Attention



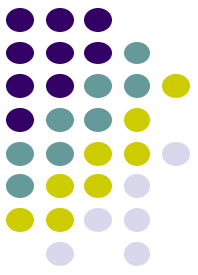


Image Captioning using spatial features and Attention



A woman is throwing a frisbee in a park.



A dog is standing on a hardwood floor.



A stop sign is on a road with a mountain in the background.



A little girl sitting on a bed with a teddy bear.

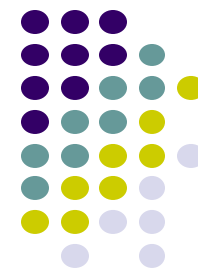


A group of people sitting on a boat in the water.



A giraffe standing in a forest with trees in the background.

General attention layer



Features	$z_{0,0}$	$z_{0,1}$	$z_{0,2}$
	$z_{1,0}$	$z_{1,1}$	$z_{1,2}$
	$z_{2,0}$	$z_{2,1}$	$z_{2,2}$

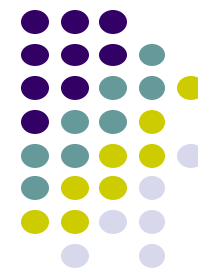
h

Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

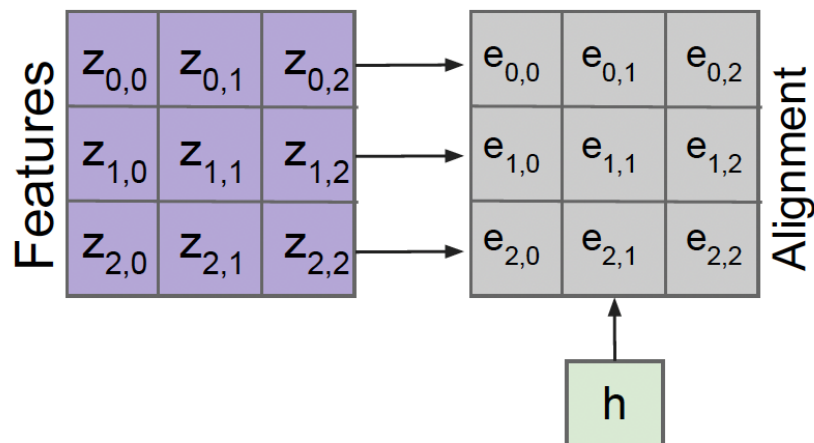
Query: $\mathbf{h}$ (shape: D)

General attention layer



Operations:

Alignment: $e_{i,j} = f_{\text{att}}(h, z_{i,j})$

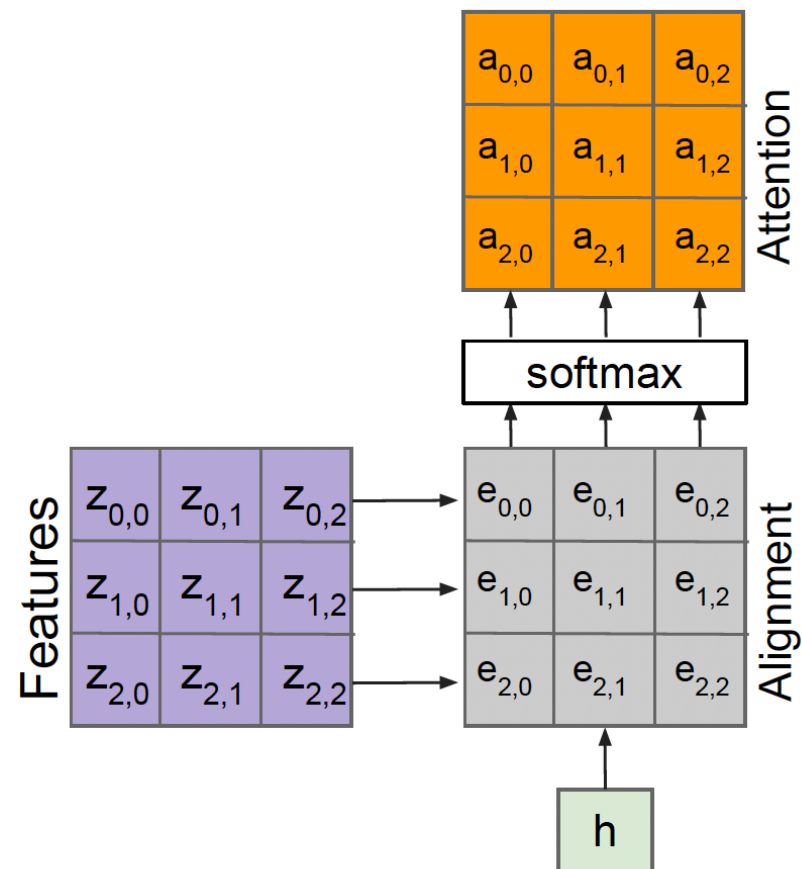
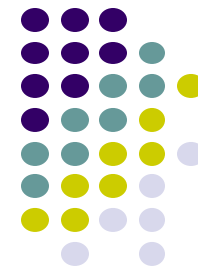


Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

General attention layer



Operations:

Alignment: $e_{i,j} = f_{\text{att}}(h, z_{i,j})$

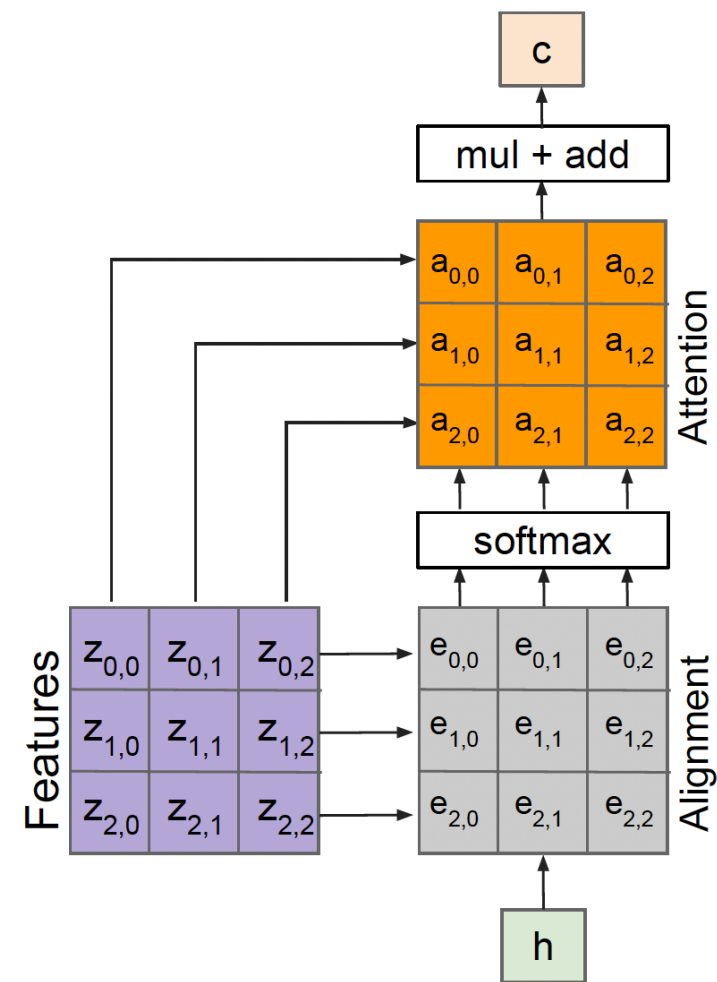
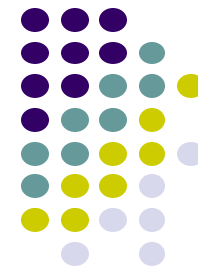
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Inputs:

Features: $\mathbf{z}$ (shape: $H \times W \times D$)

Query: $\mathbf{h}$ (shape: D)

General attention layer



Outputs:

context vector: c (shape: D)

Operations:

Alignment: $e_{i,j} = f_{\text{att}}(h, z_{i,j})$

Attention: $a = \text{softmax}(e)$

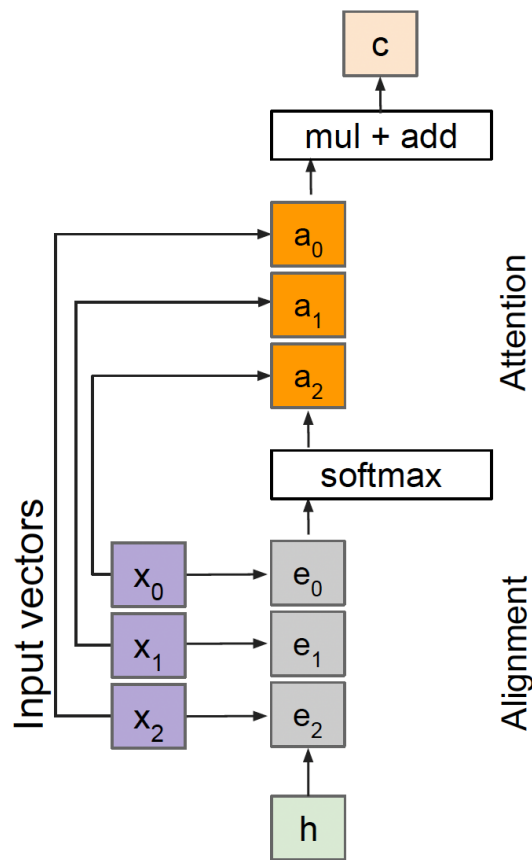
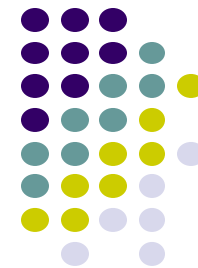
Output: $c = \sum_{i,j} a_{i,j} z_{i,j}$

Inputs:

Features: z (shape: H x W x D)

Query: h (shape: D)

General attention layer



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $\mathbf{e}_i = f_{\text{att}}(h, x_i)$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $\mathbf{c} = \sum_i a_i x_i$

Inputs:

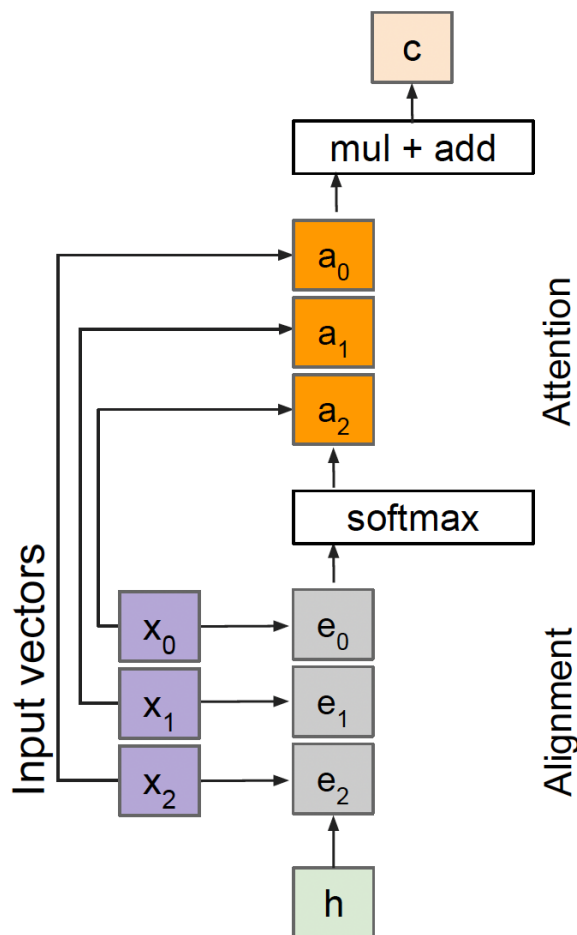
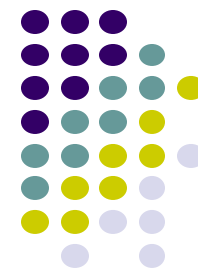
Input vectors: $\mathbf{x}$ (shape: N x D)

Query: $\mathbf{h}$ (shape: D)

Attention operation is **permutation invariant**.

- Doesn't care about ordering of the features
- Stretch $H \times W = N$ into N vectors

General attention layer



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $\mathbf{e}_i = \mathbf{h} \cdot \mathbf{x}_i$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

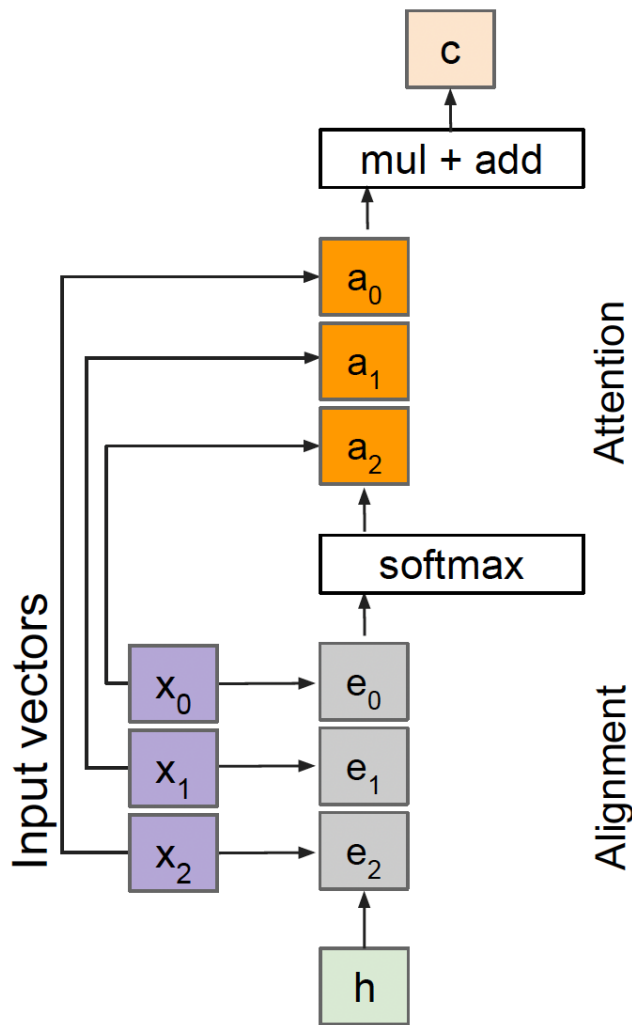
Output: $\mathbf{c} = \sum_i \mathbf{a}_i \mathbf{x}_i$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Query: $\mathbf{h}$ (shape: D)

General attention layer



Outputs:

context vector: $\mathbf{c}$ (shape: D)

Operations:

Alignment: $\mathbf{e}_i = \mathbf{h} \cdot \mathbf{x}_i / \sqrt{D}$

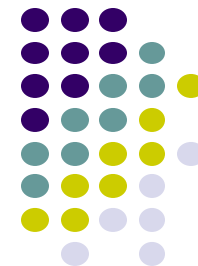
Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $\mathbf{c} = \sum_i a_i \mathbf{x}_i$

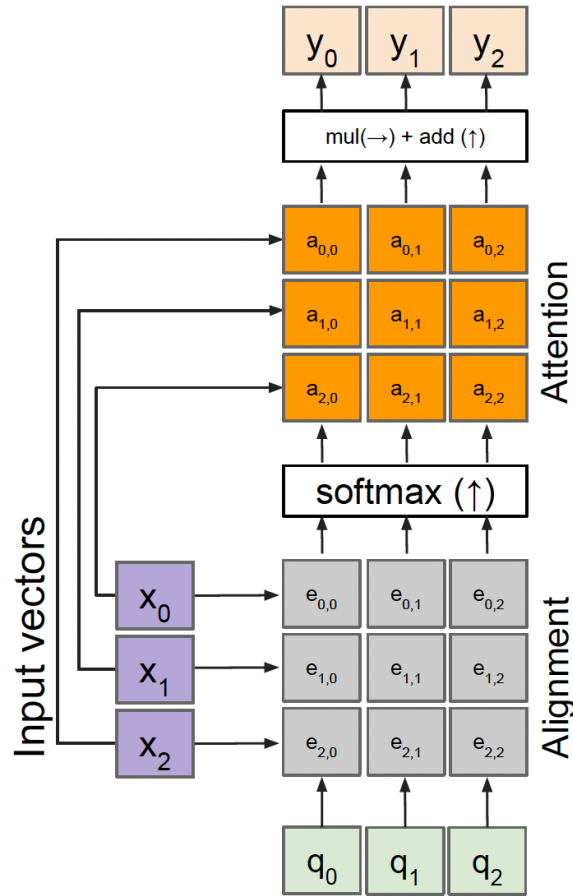
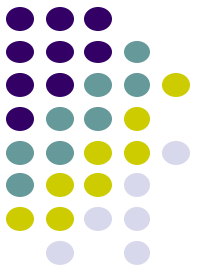
Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Query: $\mathbf{h}$ (shape: D)



General attention layer



Outputs:

context vectors: $\mathbf{y}$ (shape: D)

Operations:

Alignment: $e_{i,j} = q_j \cdot x_i / \sqrt{D}$

Attention: $\mathbf{a} = \text{softmax}(\mathbf{e})$

Output: $y_j = \sum_i a_{i,j} x_i$

Multiple query vectors

- each query creates a new output context vector

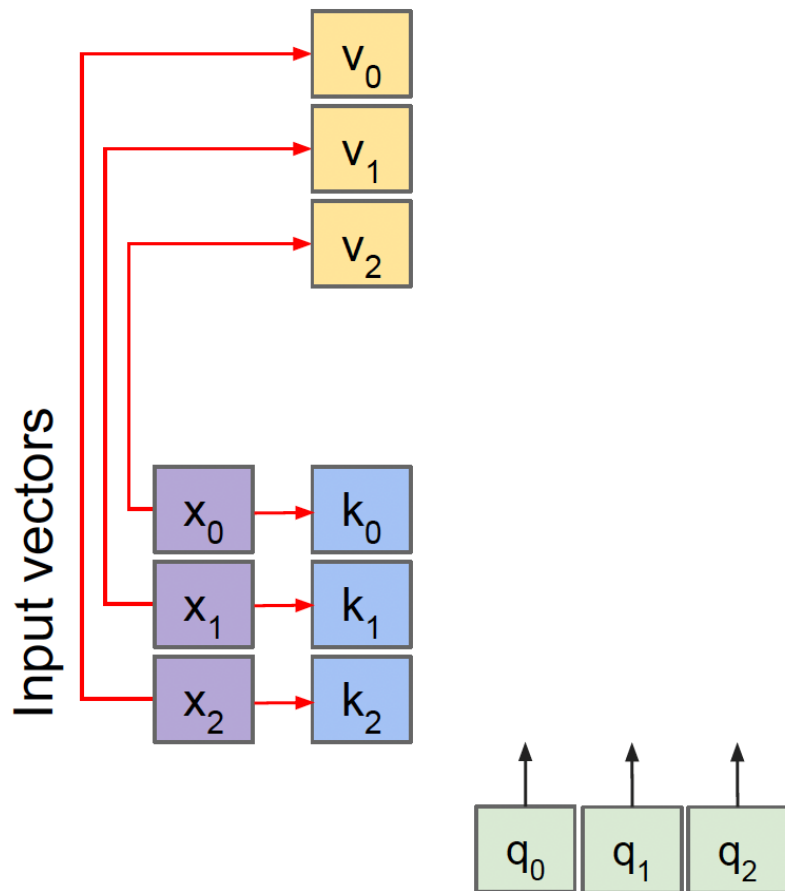
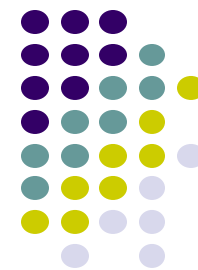
Multiple query vectors

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times D$)

General attention layer



Operations:

Key vectors: $\mathbf{k} = \mathbf{x}\mathbf{W}_k$

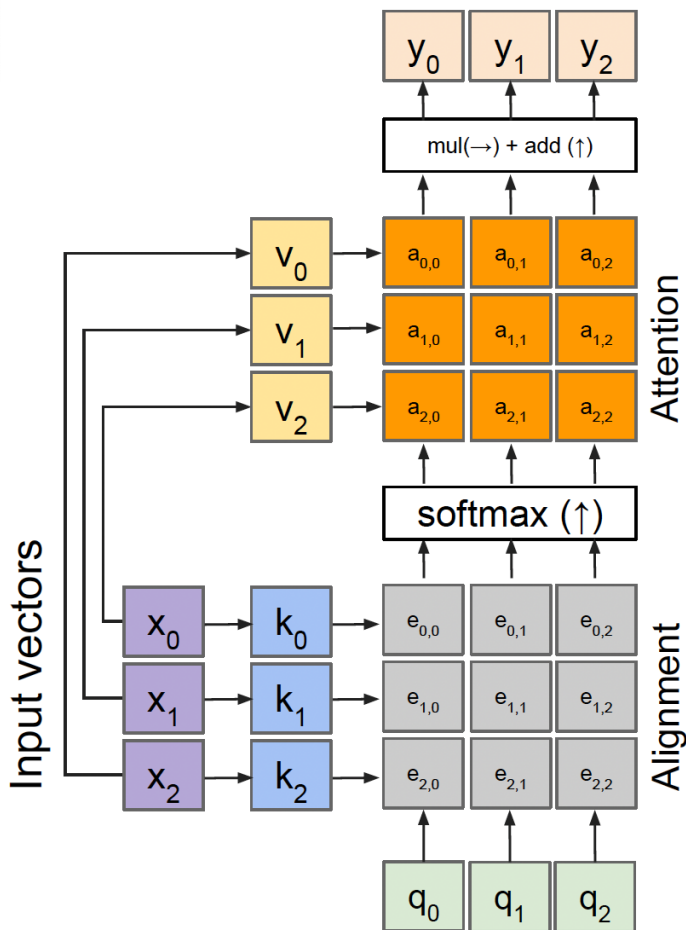
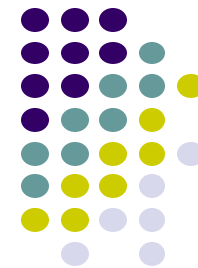
Value vectors: $\mathbf{v} = \mathbf{x}\mathbf{W}_v$

Inputs:

Input vectors: $\mathbf{x}$ (shape: $N \times D$)

Queries: $\mathbf{q}$ (shape: $M \times D_k$)

General attention layer



Outputs:

context vectors: y (shape: D_v)

Operations:

Key vectors: $k = xW_k$

Value vectors: $v = xW_v$

Alignment: $e_{i,j} = q_j \cdot k_i / \sqrt{D}$

Attention: $a = \text{softmax}(e)$

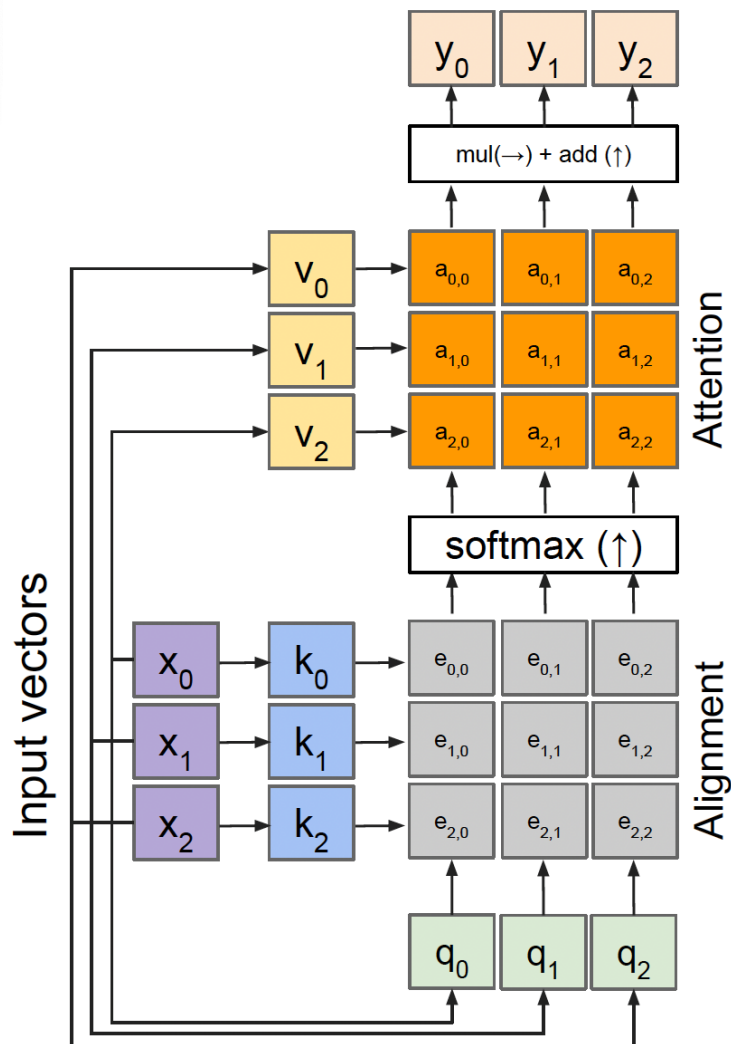
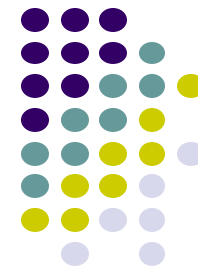
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: x (shape: $N \times D$)

Queries: q (shape: $M \times D_k$)

Self attention mechanism



Outputs:

context vectors: y (shape: D_v)

Operations:

Key vectors: $k = xW_k$

Value vectors: $v = xW_v$

Query vectors: $q = xW_q$

Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$

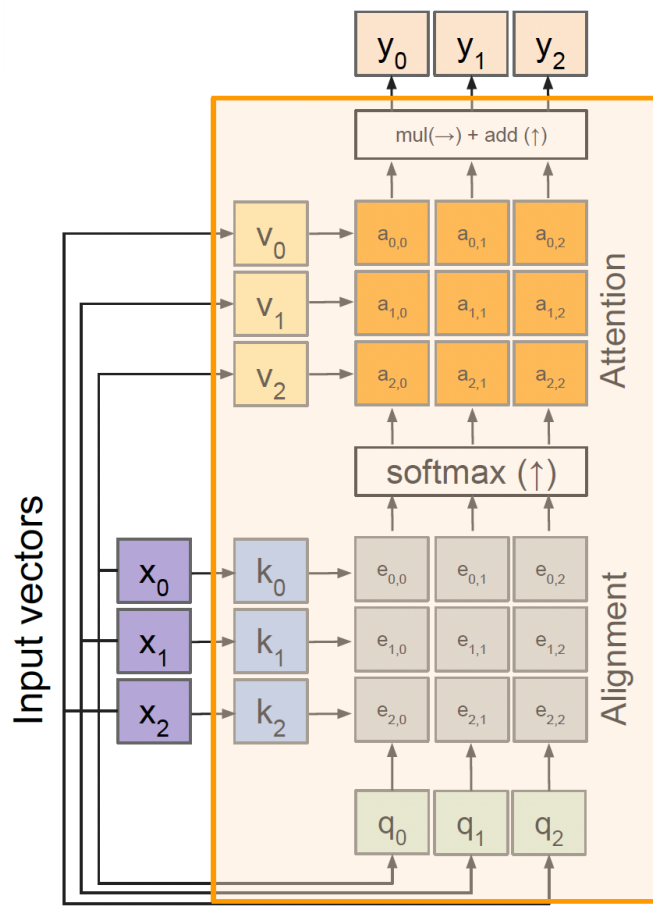
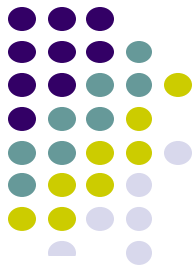
Attention: $a = \text{softmax}(e)$

Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

Input vectors: x (shape: $N \times D$)

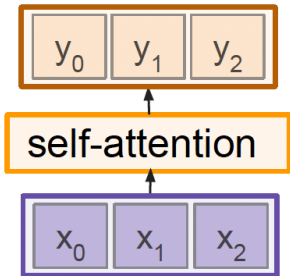
Self attention mechanism



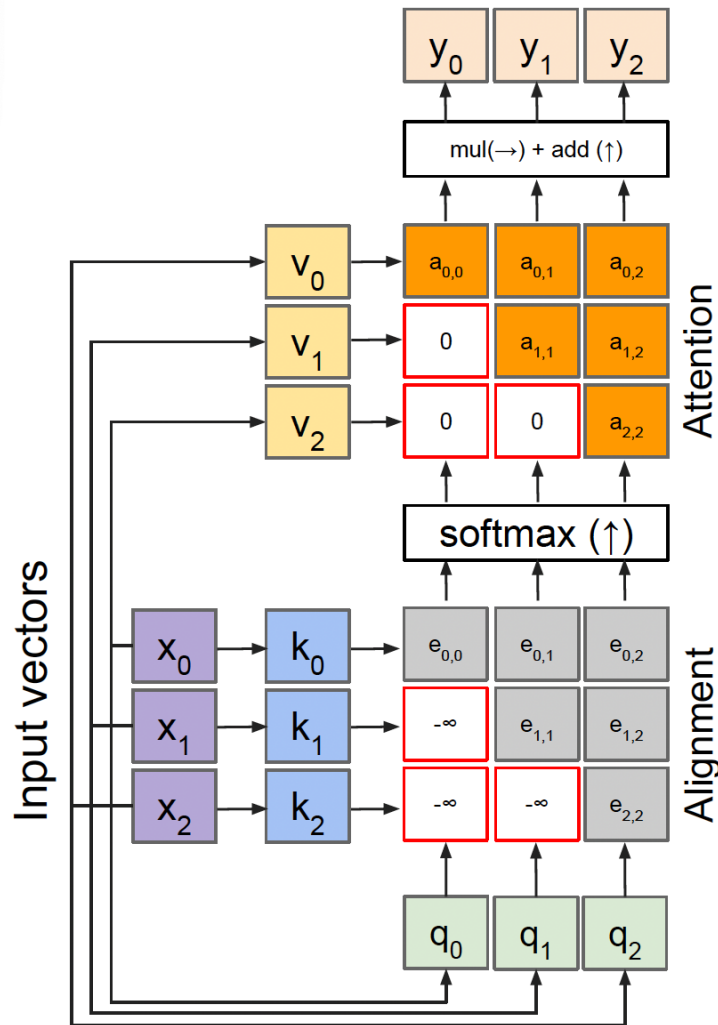
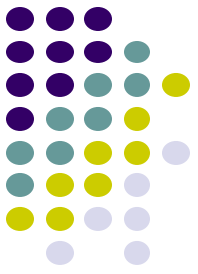
Outputs:
context vectors: y (shape: D_v)

Operations:
Key vectors: $k = xW_k$
Value vectors: $v = xW_v$
Query vectors: $q = xW_q$
Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$
Attention: $a = \text{softmax}(e)$
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:
Input vectors: x (shape: $N \times D$)



Masked Self attention mechanism



Outputs:

context vectors: y (shape: D_v)

Operations:

Key vectors: $k = xW_k$

Value vectors: $v = xW_v$

Query vectors: $q = xW_q$

Alignment: $e_{i,j} = q_i \cdot k_j / \sqrt{D}$

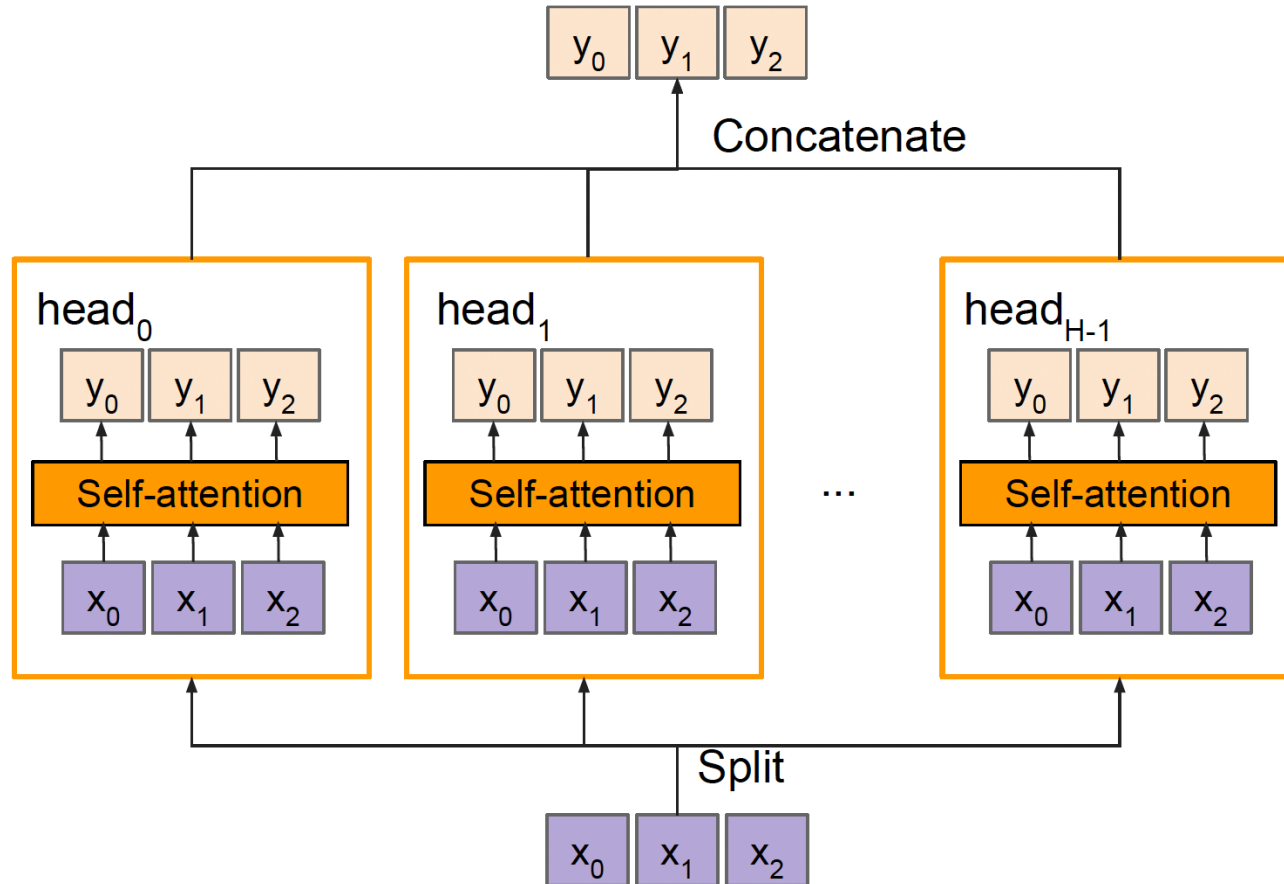
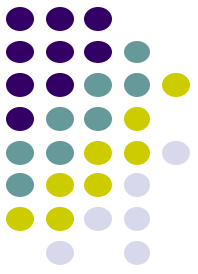
Attention: $a = \text{softmax}(e)$

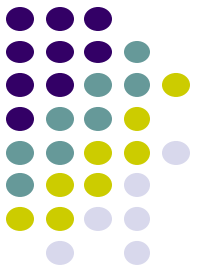
Output: $y_j = \sum_i a_{i,j} v_i$

Inputs:

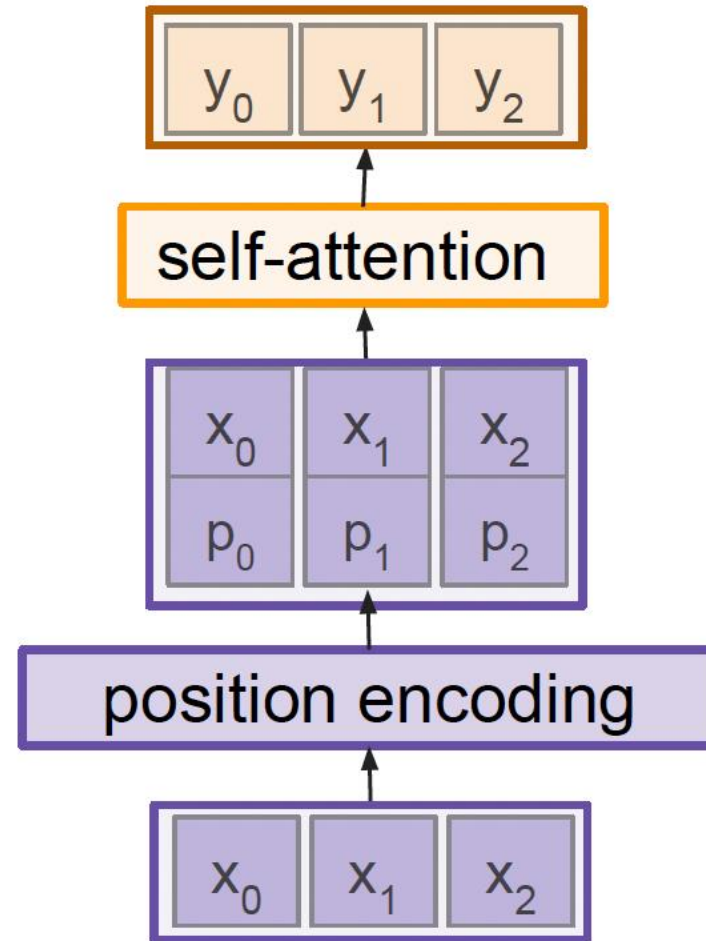
Input vectors: x (shape: $N \times D$)

Multi-head attention mechanism

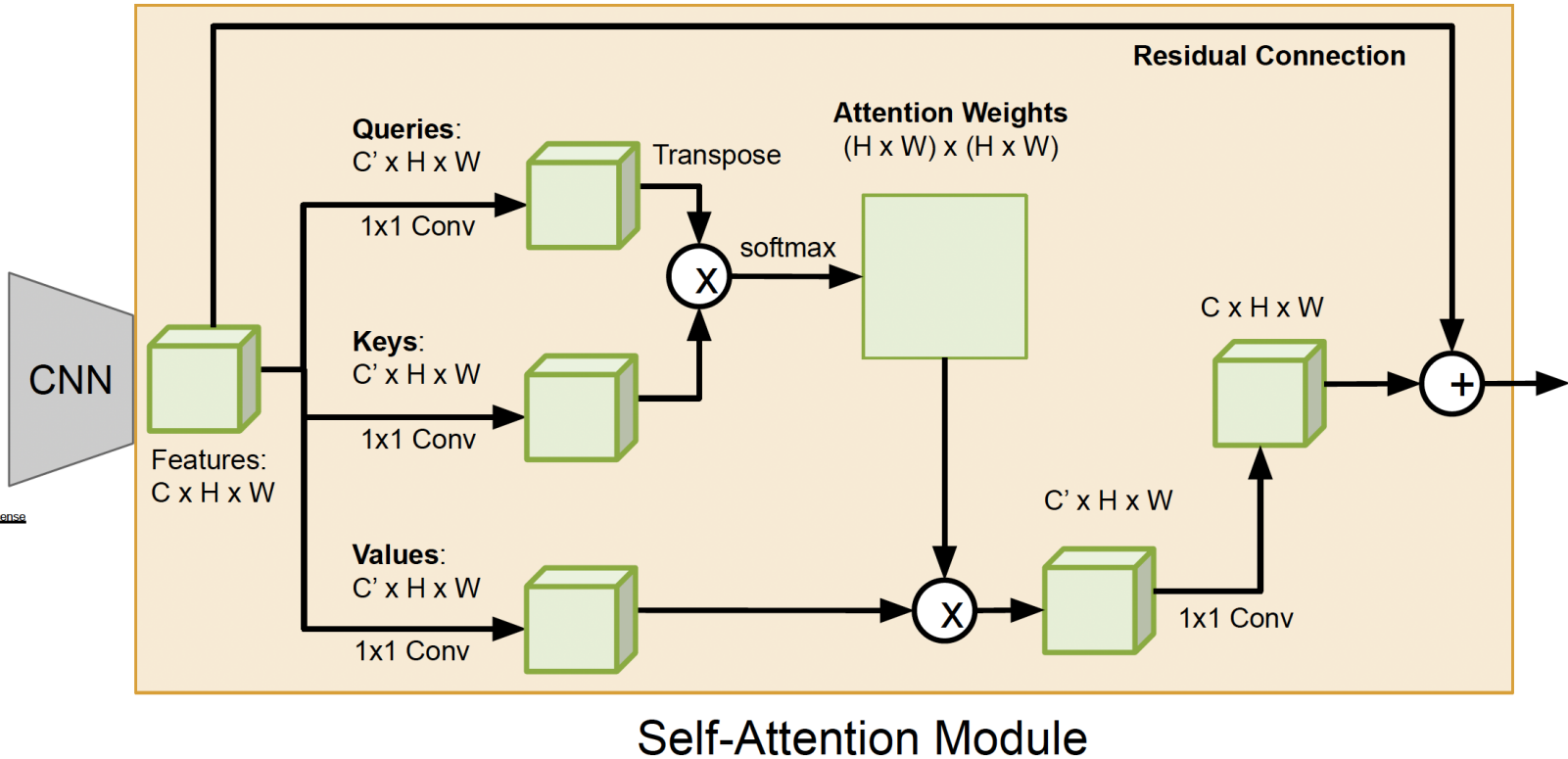
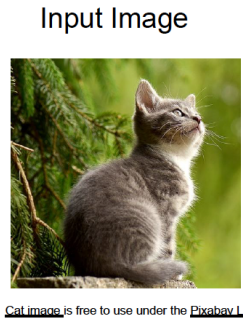




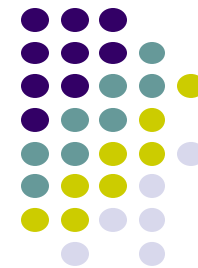
Positional encoding



CNN example with self attention layer



Transformer



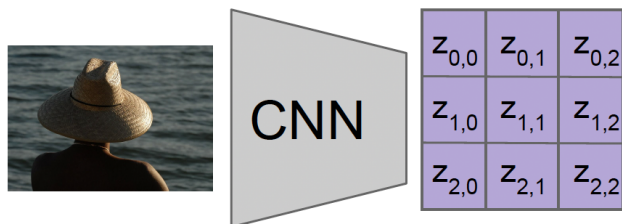
Input: Image I

Output: Sequence $\mathbf{y} = y_1, y_2, \dots, y_T$

Encoder: $\mathbf{c} = T_w(\mathbf{z})$

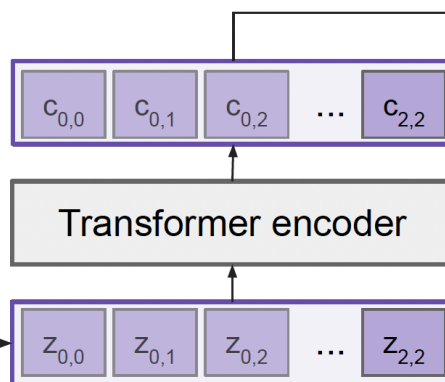
where $\mathbf{z}$ is spatial CNN features

$T_w(\cdot)$ is the transformer encoder



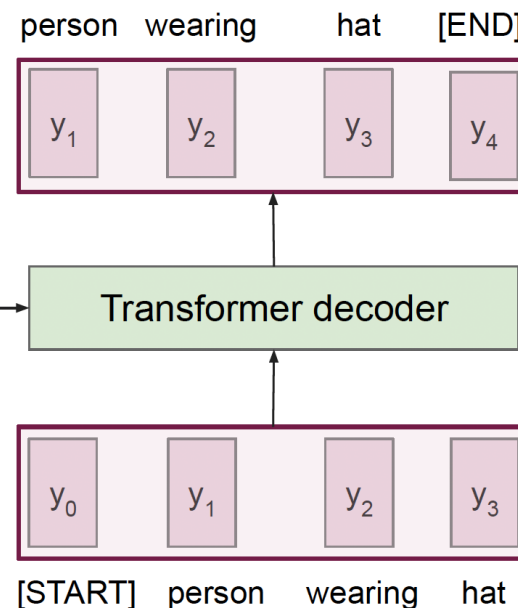
Features:
 $H \times W \times D$

Extract spatial
features from a
pretrained CNN

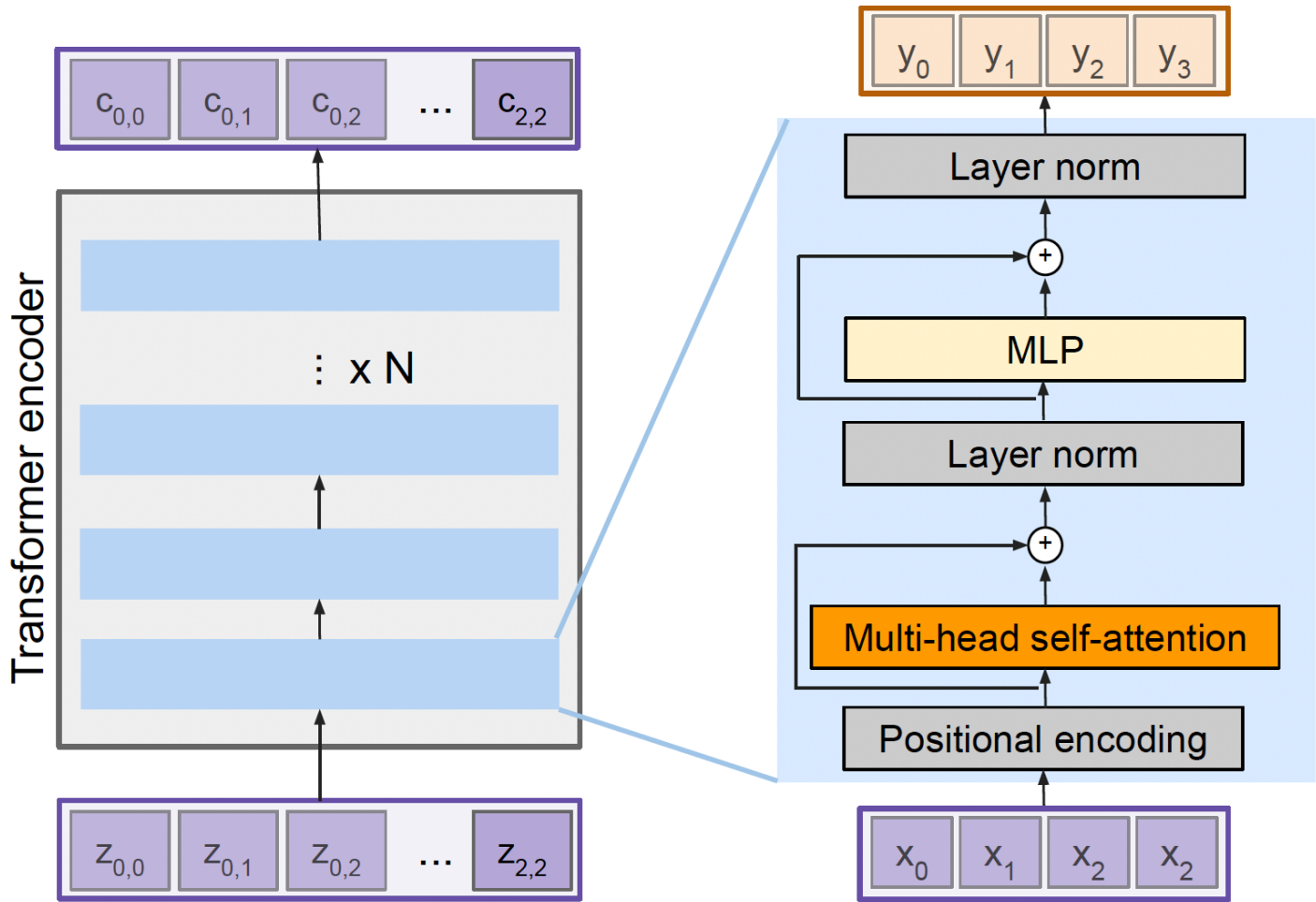
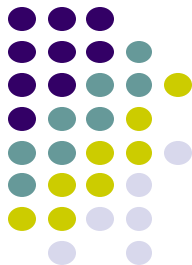


Decoder: $y_t = T_D(y_{0:t-1}, \mathbf{c})$

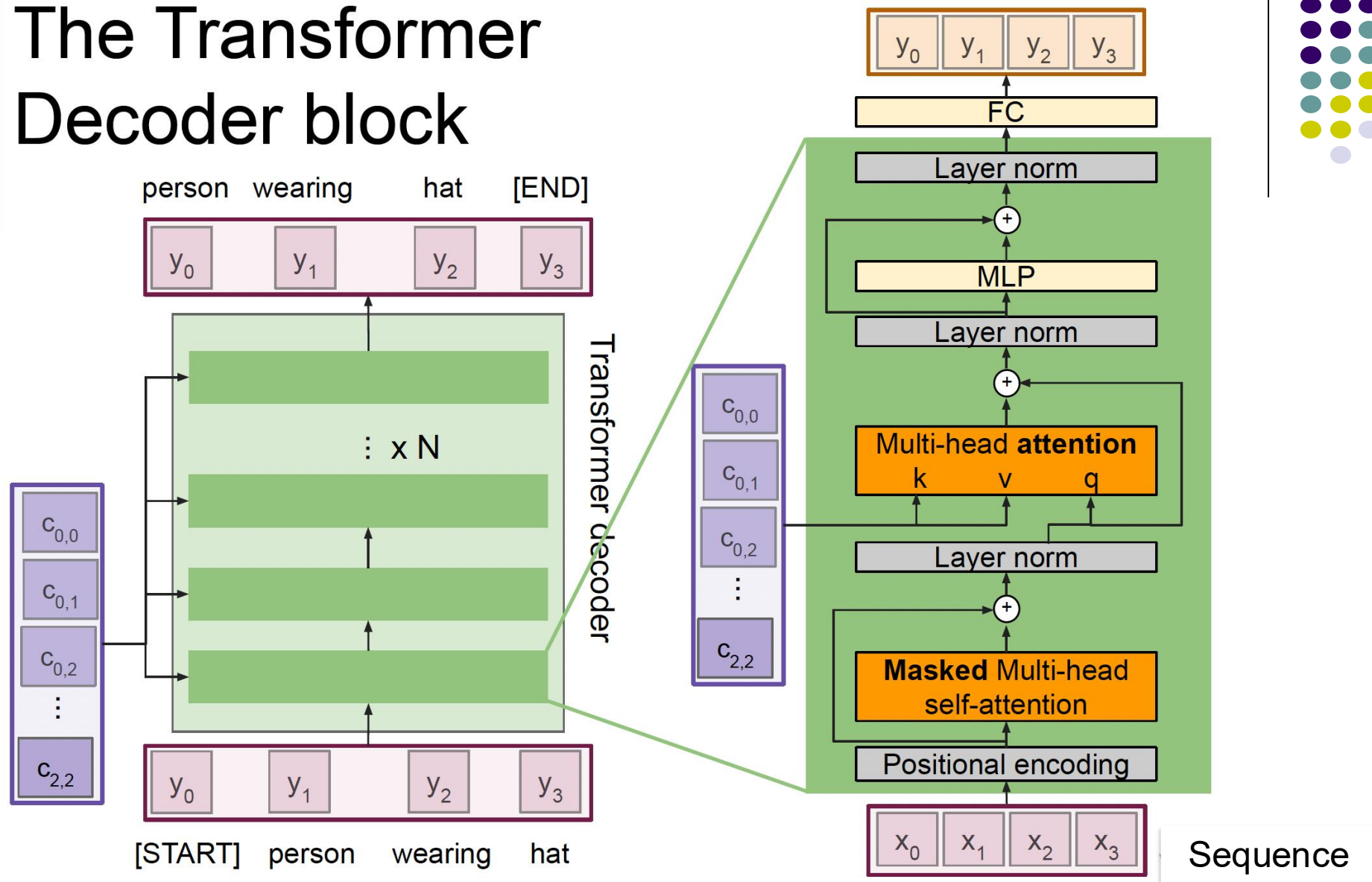
where $T_D(\cdot)$ is the transformer decoder



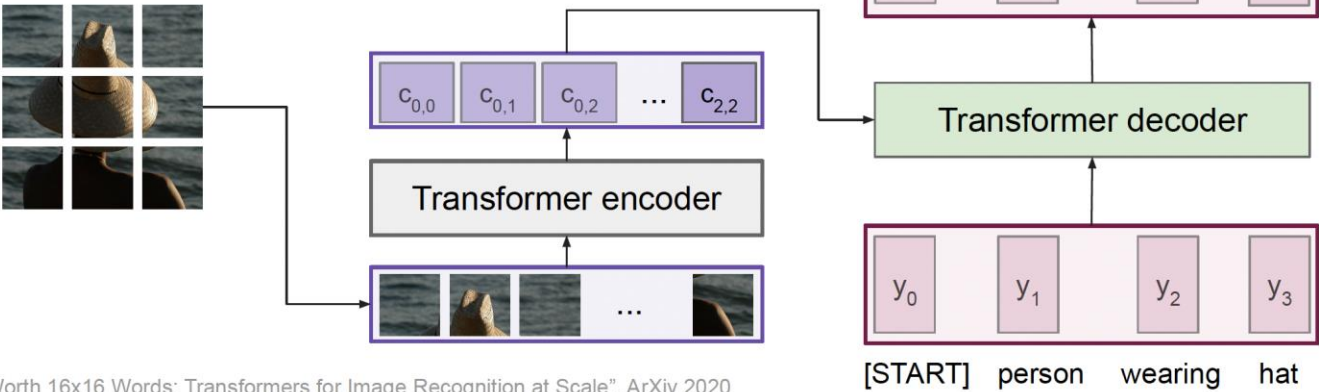
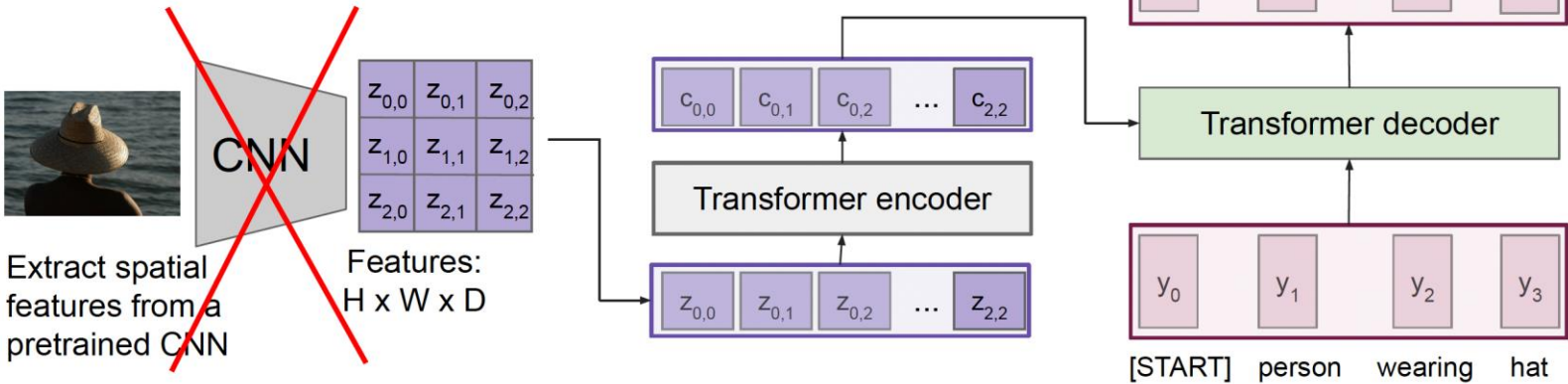
Transformer encoder



The Transformer Decoder block



Vision Transformer (ViT)



Dosovitskiy et al, "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale", ArXiv 2020
[Colab link](#) to an implementation of vision transformers

